Kendrick Kwiwongbaye

CSCI 430,530

Sarnath Ramnath

10/06/2026

Process Client's Order

Business Process

The clerk (a trusted actor) asks to process a client's order and gives the client's ID. If the client is found, the system shows the items in the client's wishlist one at a time. For each product, the clerk says whether it is being ordered and in what quantity, and the system corrects the wishlist: an item that is not being ordered is removed, an item ordered at the same quantity is left as is, and an item ordered at a different quantity has its quantity changed. When the end of the wishlist is reached, the system creates the order from the corrected wishlist. Each request is filled from stock as far as possible; any request, or part of a request, that cannot be filled is added to that product's waitlist. The total cost of the filled items is debited to the client's account. No payment is involved.

Classes Used by This Feature

This feature uses the Stage 1 classes (Warehouse, ClientList, ProdList, Client, Product, WishlistItem, WaitlistItem) and adds one new class, Order. The user interface talks only to Warehouse.

Class	**Responsibilities added or used for this feature**
Warehouse (facade)	Coordinates getWishlist(clientId), removeFromWishlist(clientId, productId), changeWishlistQuantity(clientId, productId, qty), placeOrder(clientId), and the query methods getClient, getProduct, getWaitlist, getOrders.
ClientList	search(clientId): returns the Client or null.
ProdList	search(productId): returns the Product or null.
Client	Holds balance, wishlist (WishlistItems), and orders. Uses getWishlist(), findWishlistItem(productId), removeWishlistItem(productId), clearWishlist(), debit(amount), addOrder(order), and getOrders().
WishlistItem	Holds a Product reference and quantity. Uses getProduct(), getQuantity(), and setQuantity(qty).
Product	Holds price, stock, and waitlist. Uses getStock(), getPrice(), reduceStock(qty), addToWaitlist(client, qty), and getWaitlist().
WaitlistItem	Holds a client reference and the unfilled quantity. Uses getClient() and getQuantity().
Order (new)	Holds order ID, date, client, and order lines (product, quantity filled, unit price). Uses addItem(product, qty, price), getItems(), and getTotal().

Question 1:

Query Implementation

Processing an order can change five kinds of stored data: the client's balance, the client's wishlist, each product's stock, each product's waitlist, and the client's list of orders. The queries below are run before and after processing the order so the results can be compared.

Data Stored In The Warehouse System That May Be Modified	Query Before Testing The Operation	Query After Testing The Operation	How The Before And After Queries Verify The Operation Was Done Correctly
client details by client ID	Run Query Client Details and note the client's ID, name, address, and current credit/debit balance.	Run Query Client Details again and note the updated balance.	Verifies that the correct client was processed and that the total cost of the filled items was debited to the client's balance. No payment is recorded.
the client's wishlist	Run Query Client Wishlist and note every product and requested quantity.	Run Query Client Wishlist again after the order has been created and the processed WishlistItems have been cleared.	Verifies the clerk's corrections. Items not ordered are removed during review, changed quantities are recorded correctly, and processed ordered items are removed after order creation as a design choice to prevent duplicate processing.
product details	Run Query Product Details for each product involved in the order and note product ID, product name, quantity in stock, and unit price.	Run Query Product Details again for the same products after the order has been processed.	Verifies that stock decreased by exactly the quantity filled and never falls below zero. Products not ordered remain unchanged.
each product's waitlist	Run Query Product Waitlist for each product involved in the client's order and note the existing entries.	Run Query Product Waitlist again after processing the order.	Verifies that any request, or part of a request, that could not be filled was added to the correct product's waitlist with the correct unfilled quantity.
the client's orders	Run Query Client Orders and note the client's existing orders or verify that the new order does not yet exist.	Run Query Client Orders again after processing and locate the newly created order.	Verifies that a new order was created for the correct client and contains the quantities that were actually filled.
the order total	Run Query Client Details and record the client's balance before processing the order.	Run Query Client Orders to obtain the new order total, then run Query Client Details to obtain the updated balance.	Verifies that the total equals the sum of filled quantity × unit price and that this amount was debited to the client's balance.
unfilled quantities	Run Query Client Wishlist, Query Product Details, and Query Product Waitlist to record requested quantities, current stock, and existing waitlist entries.	Run Query Client Orders, Query Product Details, and Query Product Waitlist after processing and compare the filled quantities, updated stock, and waitlist entries.	For each product, quantity filled + quantity waitlisted equals quantity ordered, and quantity filled equals old stock − new stock. Only the unfilled part is waitlisted.

Example test. Client C1 has a balance of $0.00 and wishlist P1 × 5, P2 × 2, P3 × 1. Stock and prices: P1 = 10 at $2.00, P2 = 1 at $10.00, P3 = 4 at $5.00. The clerk orders P1 at the same quantity, changes P2 to 3, and does not order P3.

Query	Expected result after processing
Query Client Orders (C1)	One new order: P1 × 5 at $2.00 = $10.00; P2 × 1 at $10.00 = $10.00. Total $20.00.
Query Client Details (C1)	Balance $20.00 (was $0.00).
Query Product Details	P1 stock 5 (was 10); P2 stock 0 (was 1); P3 stock 4 (unchanged).
Query Product Waitlist (P2)	New entry: C1, quantity 2.
Query Client Wishlist (C1)	No processed order items remain. P3 was removed when the clerk chose not to order it; P1 and P2 were cleared after the order was created to prevent duplicate processing.

Question 2:

Detailed Use Case For The Main Process Client's Order

Actor: Clerk (trusted). Preconditions: clients and products exist; the client may have items in a wishlist. Postconditions: filled quantities are stored in an order, stock and waitlists are updated, the client is debited for filled quantities only, and processed WishlistItems are removed after the order has been created so the same request is not processed twice.

Actor	System Response
1. Request to process a client's order.	2. Ask for client ID.
3. Enter client ID.	4. Search for the client. If the client does not exist, display a message and exit the operation.
	5. Retrieve the client's wishlist. If the wishlist is empty, display a message and exit the operation.
	6. Display the product ID, product name, requested quantity, quantity in stock, and unit price of the next wishlist item. Ask whether the item is being ordered.
7. Enter yes or no.	8. If no, remove the item from the wishlist and go to step 12. If yes, ask for the quantity, showing the current wishlist quantity as the default.
9. Enter a quantity or accept the default.	10. Validate that the quantity is a positive whole number. If invalid, display an error and ask again.
	11. If the quantity is the same as the wishlist quantity, leave the WishlistItem unchanged. If different, change the WishlistItem quantity to the new quantity.
	12. If there are more items in the wishlist, repeat steps 6–11 for the next item. Continue until the end of the wishlist is reached.
	13. If the corrected wishlist is empty, display "No items were ordered" and exit.
	14. Create a new order for the client with a unique order ID and the current date, using the corrected wishlist.
	15. For each corrected WishlistItem, compute filledQty = min(orderedQty, quantityInStock), reduce product stock by filledQty,

Actor	System Response
	and add the filled quantity and unit price to the order.
	16. Compute unfilledQty = orderedQty − filledQty. If unfilledQty > 0, add a WaitlistItem for that client and quantity to the product's waitlist.
	17. After the order has been created from the corrected wishlist, call Client.clearWishlist() to clear the processed wishlist. This is a design choice to prevent the same requested items from being processed again; it is not part of the payment process.
	18. If no quantity could be filled for any product, do not store an empty order and do not debit the client. Keep the waitlist additions and display that all requested quantities were waitlisted.
	19. Otherwise, calculate the order total as the sum of filledQty × unitPrice, debit that total to the client's credit/debit balance, and store the order with the client. No payment is processed.
	20. Display the order ID, date, each filled product and quantity, unit price, line cost, total, new client balance, and any quantities placed on waitlists.
	21. Display "Order processed" and exit the operation.

Detailed Use Cases for Queries Needed to Verify the Feature

Query Client Details

Actor	System Response
1. Request client details.	2. Ask for client ID.
3. Enter client ID.	4. Search for the client. If the client does not exist, display a message and exit.
	5. Display the client ID, name, address, phone, and current credit/debit balance.

Query Client Wishlist

Actor	System Response
1. Request to display a client's wishlist.	2. Ask for client ID.
3. Enter client ID.	4. Search for the client. If the client does not exist, display a message and exit.
	5. Retrieve the client's WishlistItems.
	6. If none remain, display "Wishlist is empty." Otherwise display each product ID, product name, and current quantity.

Query Product Details

Actor	System Response
1. Request product details.	2. Ask for product ID.
3. Enter product ID.	4. Search for the product using ProdList. If the product does not exist, display a message and exit.
	5. Display the product ID, product name, quantity in stock, and unit price.

Query Product Waitlist

Actor	System Response
1. Request to display a product's waitlist.	2. Ask for product ID.
3. Enter product ID.	4. Search for the product using ProdList. If the product does not exist, display a message and exit.
	5. Retrieve the product's waitlist.
	6. If empty, display "Waitlist is empty." Otherwise display each waitlist entry, including client ID/name and unfilled quantity.

Query Client Orders

Actor	System Response
1. Request to display a client's orders.	2. Ask for client ID.
3. Enter client ID.	4. Search for the client. If the client does not exist, display a message and exit.
	5. Retrieve the orders belonging to the client.
	6. If no orders exist, display "Client has no orders." Otherwise display each order ID, date, product lines, quantities, unit prices, line costs, and total.

Question 3:

Design a Solution: Process Client's Order

The work is split into two parts. First, UserInterface walks through the wishlist with the clerk and asks Warehouse to correct it item by item. Second, Warehouse.placeOrder(clientId) creates and fills the order from the corrected wishlist. The user interface communicates with Warehouse as the facade; collection searches remain the responsibility of ClientList and ProdList.

Step	What the system does	Responsible class.method	Design choice
1	Get the client ID from the clerk and find the client.	UserInterface → Warehouse.getWishlist(clientId) → ClientList.search(clientId)	The UI talks only to Warehouse. ClientList owns the client collection, so it performs the lookup. A null result ends the operation.

Step	What the system does	Responsible class.method	Design choice
2	Get the client's wishlist.	Client.getWishlist() returns an iterator/copy of WishlistItems	Client owns its WishlistItems. Iterating over a copy lets the system remove or change entries without breaking the traversal.
3	Display each item and ask whether it is ordered and the quantity.	UserInterface uses WishlistItem.getProduct() and getQuantity()	The UI is responsible for interaction. It validates that an entered quantity is a positive whole number before requesting a business update.
4a	Item not ordered: remove it.	Warehouse.removeFromWishlist(clientId, productId) → ClientList.search → Client.removeWishlistItem(productId)	All updates go through Warehouse. Removing a declined item implements the required wishlist correction.
4b	Item ordered with a new quantity: change it.	Warehouse.changeWishlistQuantity(clientId, productId, qty) → Client.findWishlistItem(productId) → WishlistItem.setQuantity(qty)	WishlistItem owns the requested quantity, so it is responsible for changing it.
4c	Item ordered at the same quantity.	No update call	Leaving the item unchanged avoids an unnecessary state change.
5	At the end of the wishlist, place the order.	UserInterface → Warehouse.placeOrder(clientId) → ClientList.search → Client.getWishlist()	Warehouse coordinates order creation so stock, waitlist, balance, and order changes occur in one operation. If the corrected wishlist is empty, no order is created.
6	Create the order.	Warehouse creates new Order(client, date)	Order is a distinct class because it stores its own ID, date, lines, and total and must be queried later.
7	Fill each item from stock.	Product.getStock(); Product.reduceStock(filledQty); Product.getPrice(); Order.addItem(product, filledQty, price)	filledQty = min(orderedQty, stock). Product owns stock, so only Product changes it. Unit price is copied into the order line so later price changes do not alter past orders.
8	Waitlist the unfilled part.	Product.addToWaitlist(client, unfilledQty) creates WaitlistItem	unfilledQty = orderedQty − filledQty. Only a positive unfilled quantity is added to the product's waitlist.
9	Clear the processed wishlist after order creation.	Client.clearWishlist()	This is an explicit design choice made after the order has already been created from the corrected wishlist. It prevents duplicate processing without changing the required order-building logic.
10	Compute the total and debit the client.	Order.getTotal(); Client.debit(total)	Order computes its total from filled lines. Client owns the balance. Debit changes the balance only; no payment is processed.
11	Store and return the order.	Client.addOrder(order); Warehouse returns the Order	Orders are stored with their Client. If nothing was filled, an empty order is not stored and the client is not debited.
12	Display the result.	UserInterface uses Order.getItems(), Order.getTotal(), Client balance, and waitlist results	The clerk sees the filled quantities, total, updated balance, and any waitlisted quantities.

Queries Used to Verify Process Client's Order

Query	Steps and responsible class.method	Design choice
Query Client Details	UserInterface → Warehouse.getClient(clientId) → ClientList.search(clientId); UI displays client ID, name,	Read-only query. Showing the balance allows before/after

Query	Steps and responsible class.method	Design choice
	address, phone, and balance using Client getters.	comparison of the debit.
Query Client Wishlist	UserInterface → Warehouse.getWishlist(clientId) → ClientList.search → Client.getWishlist(); UI iterates WishlistItems and displays product ID, name, and quantity.	Uses the same access path as order processing. The before/after values show removals and quantity changes; after order creation, the processed wishlist is cleared with Client.clearWishlist() by the stated design choice.
Query Product Details	UserInterface → Warehouse.getProduct(productId) → ProdList.search(productId); UI displays ID, name, price, and stock using Product getters.	ProdList owns the product collection. Stock before/after verifies the filled quantity.
Query Product Waitlist	UserInterface → Warehouse.getWaitlist(productId) → ProdList.search → Product.getWaitlist(); UI iterates WaitlistItems and displays client ID/name and quantity.	The waitlist belongs to Product. A new WaitlistItem verifies the exact unfilled quantity.
Query Client Orders	UserInterface → Warehouse.getOrders(clientId) → ClientList.search → Client.getOrders(); UI displays order ID, date, lines, and total using Order methods.	Orders are stored with the client. The new order verifies filled quantities and the total used for the debit.

Question 4:

Sequence Diagrams

Process Client’s Order

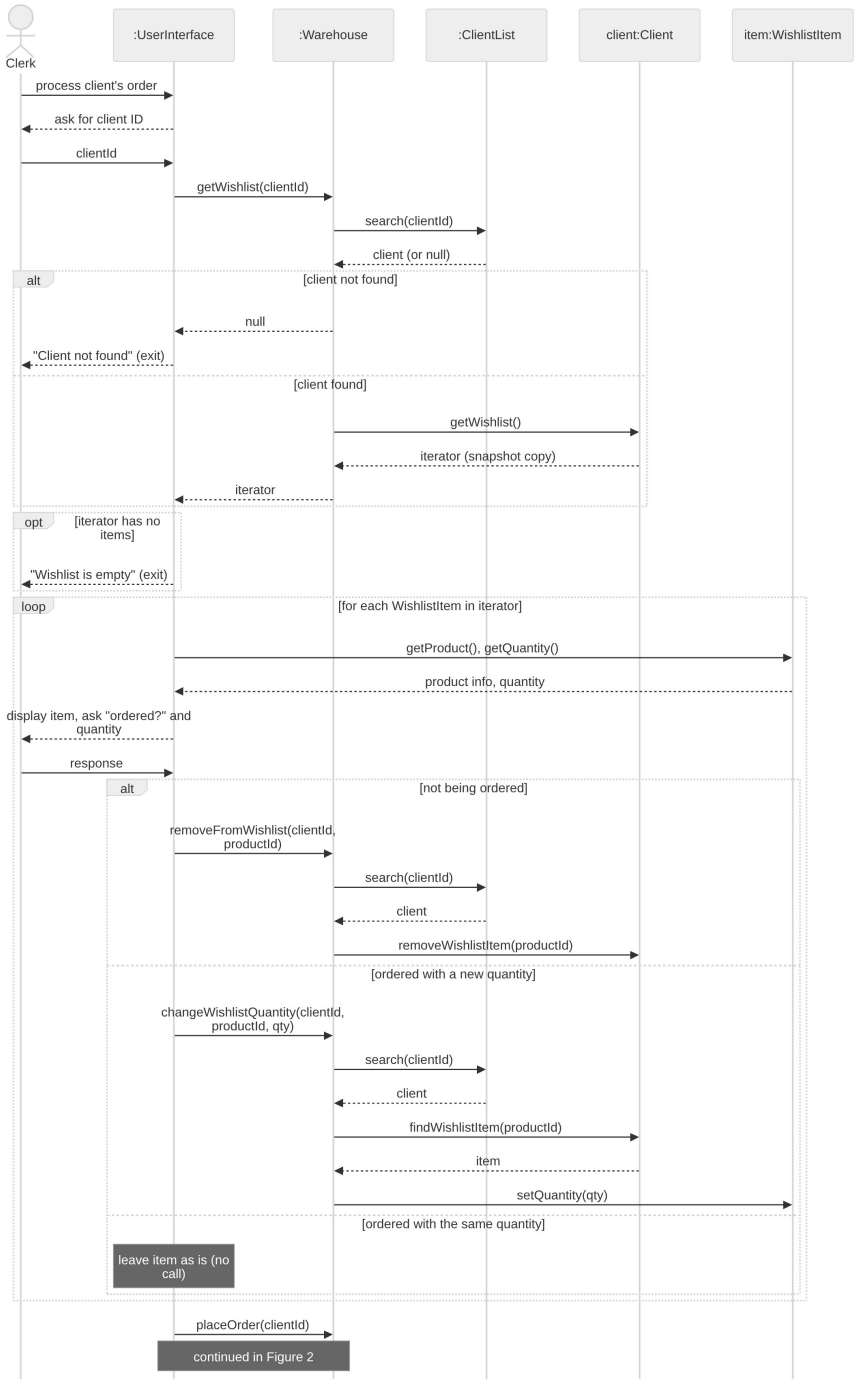


Figure 1. Process Client’s Order, part 1: reviewing and correcting the wishlist

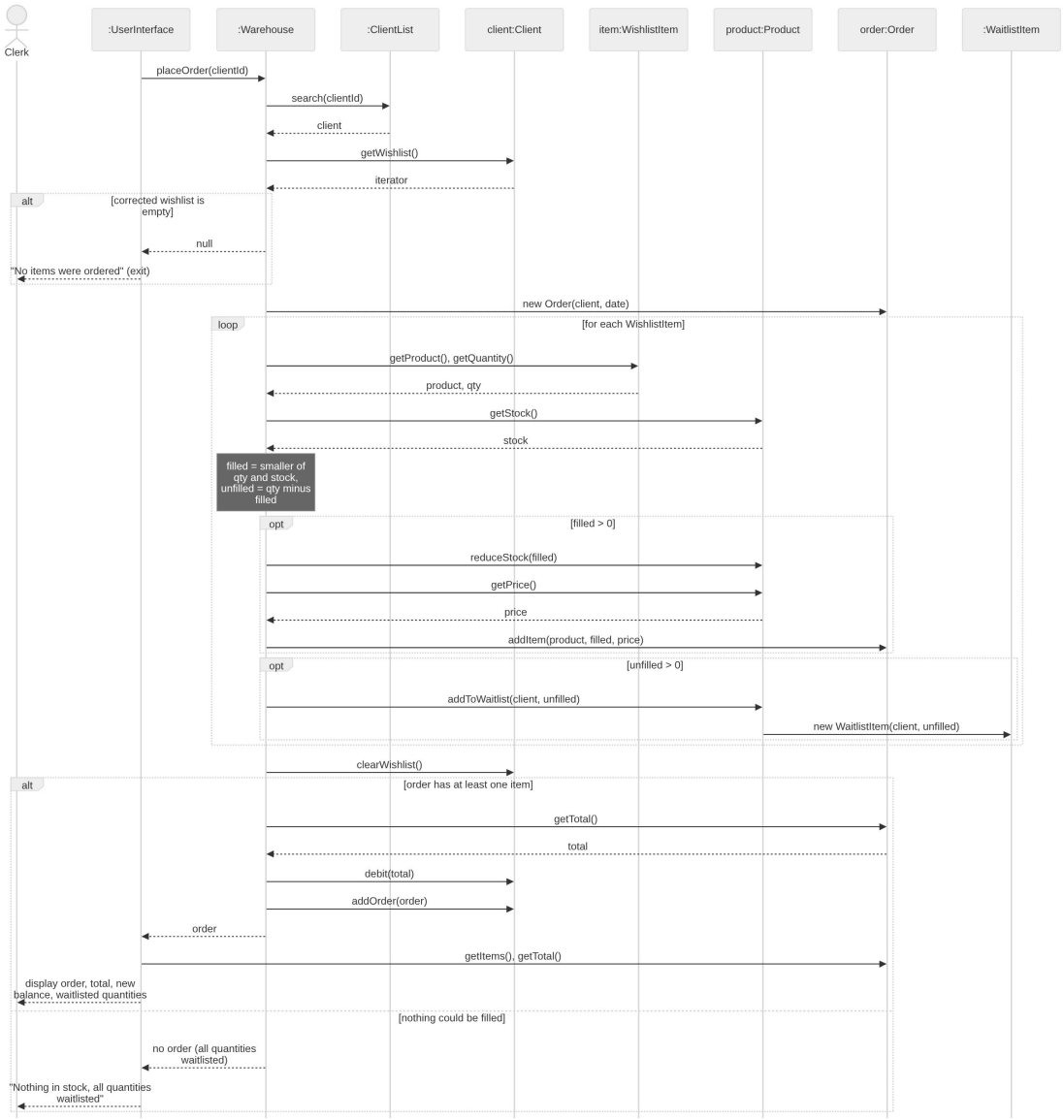


Figure 2. Process Client's Order, part 2: Warehouse.placeOrder(clientId)

Verification Queries

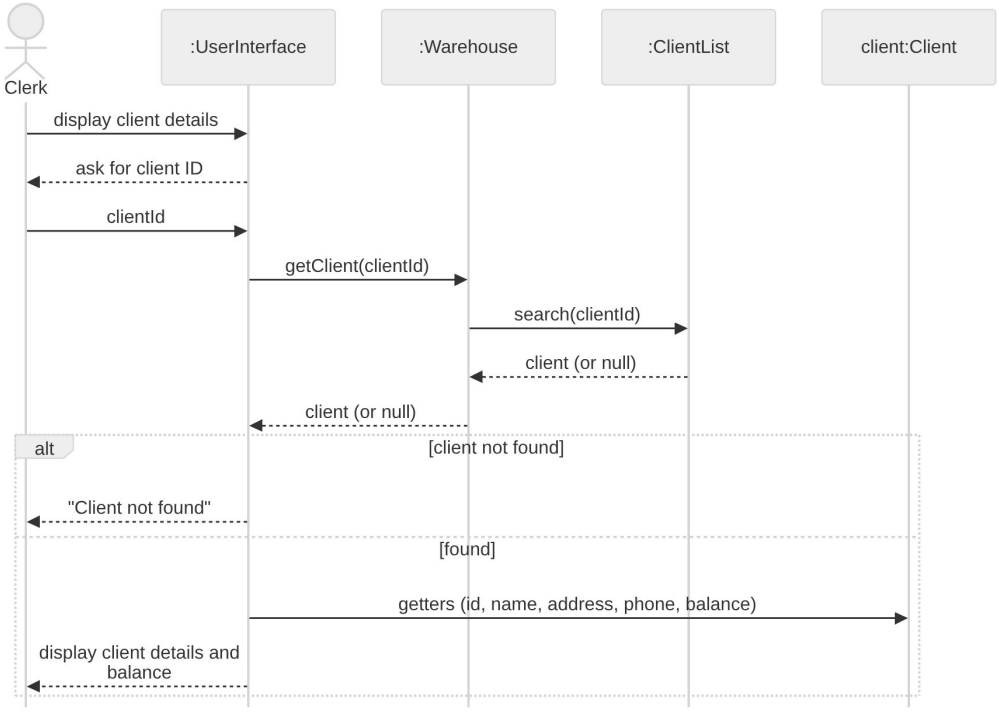


Figure 3. Query Client Details

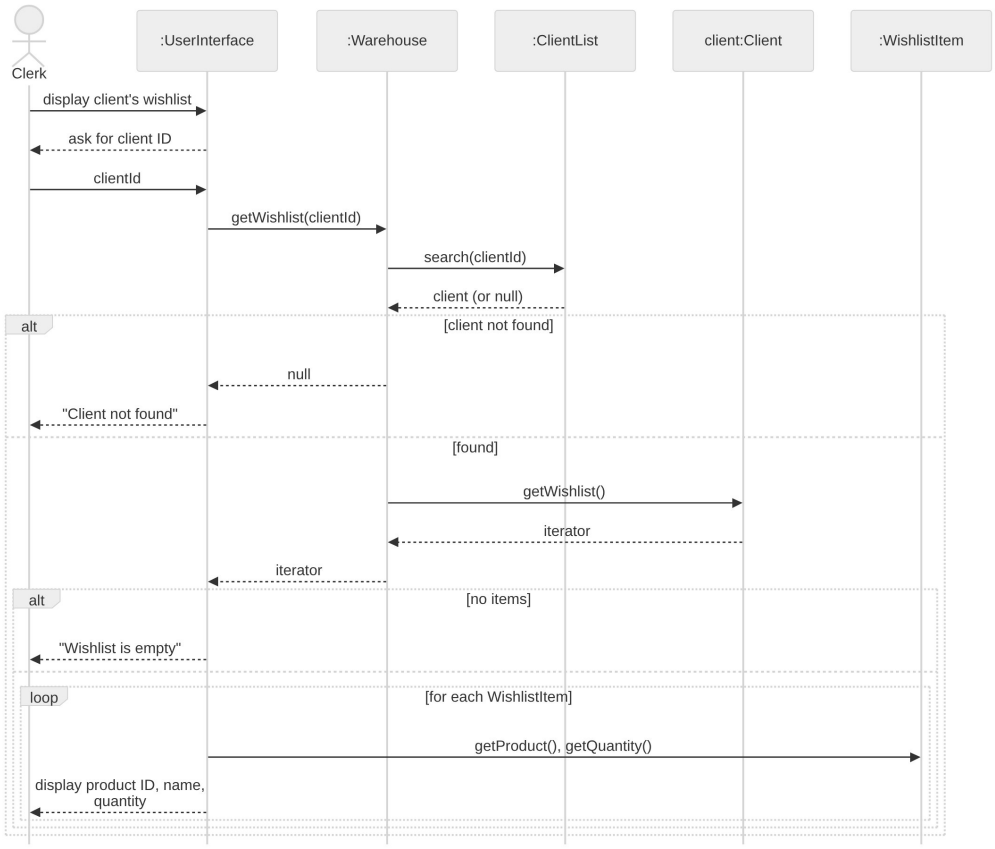


Figure 4. Query Client Wishlist

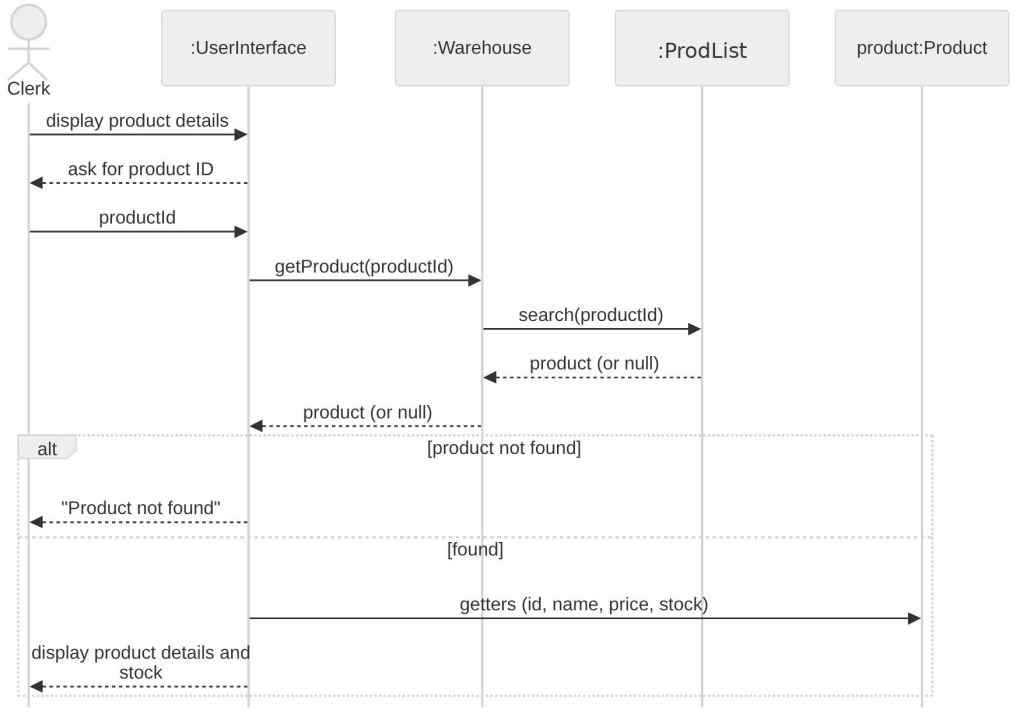


Figure 5. Query Product Details (uses ProdList)

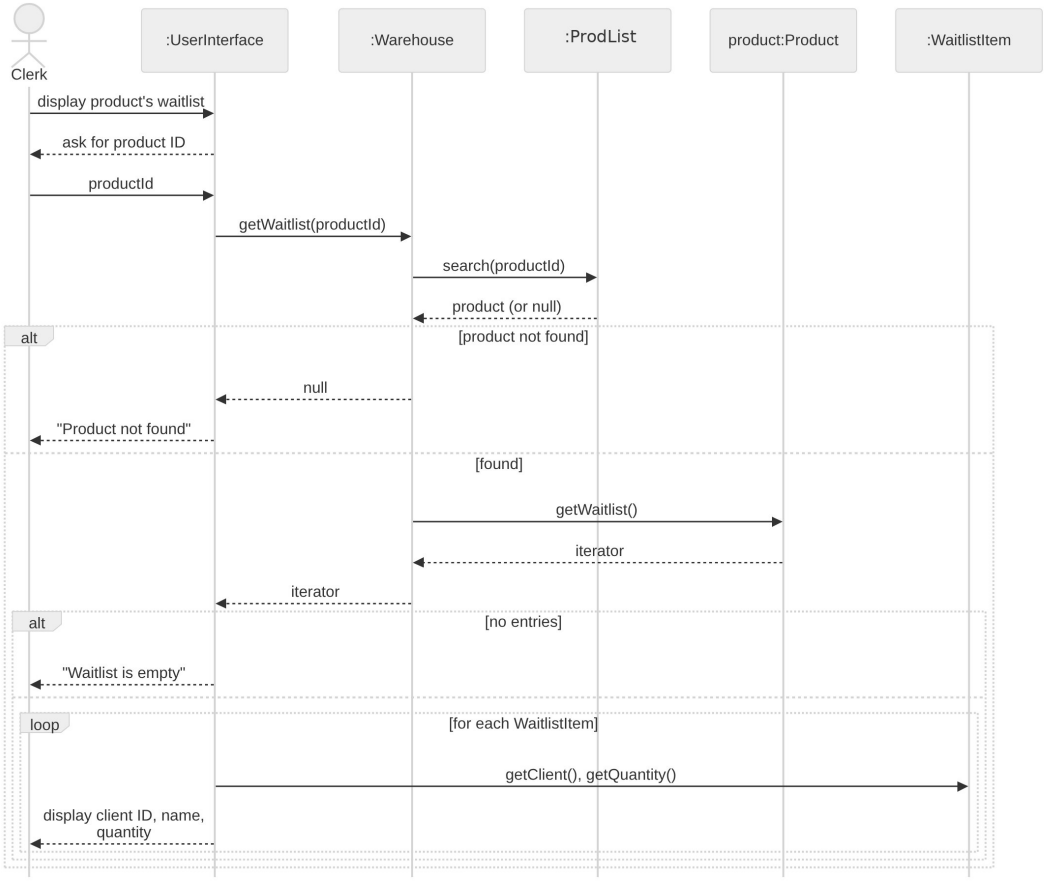


Figure 6. Query Product Waitlist (uses ProdList)

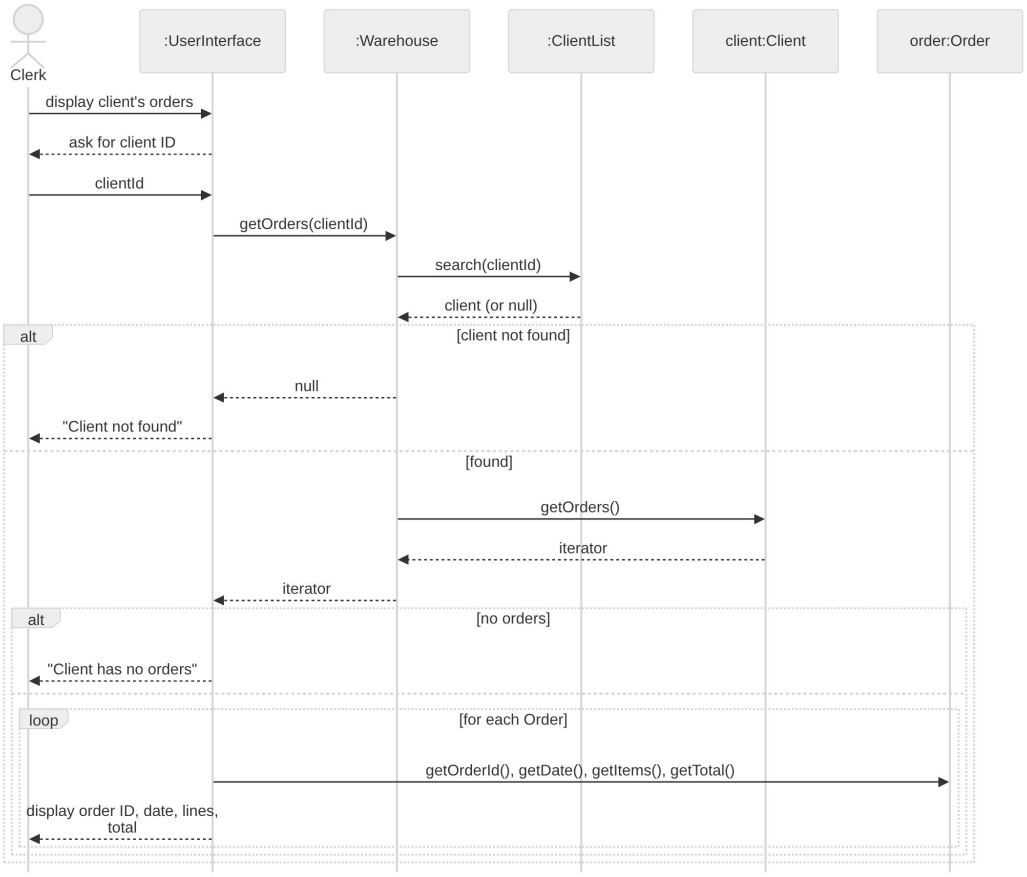


Figure 7. Query Client Orders